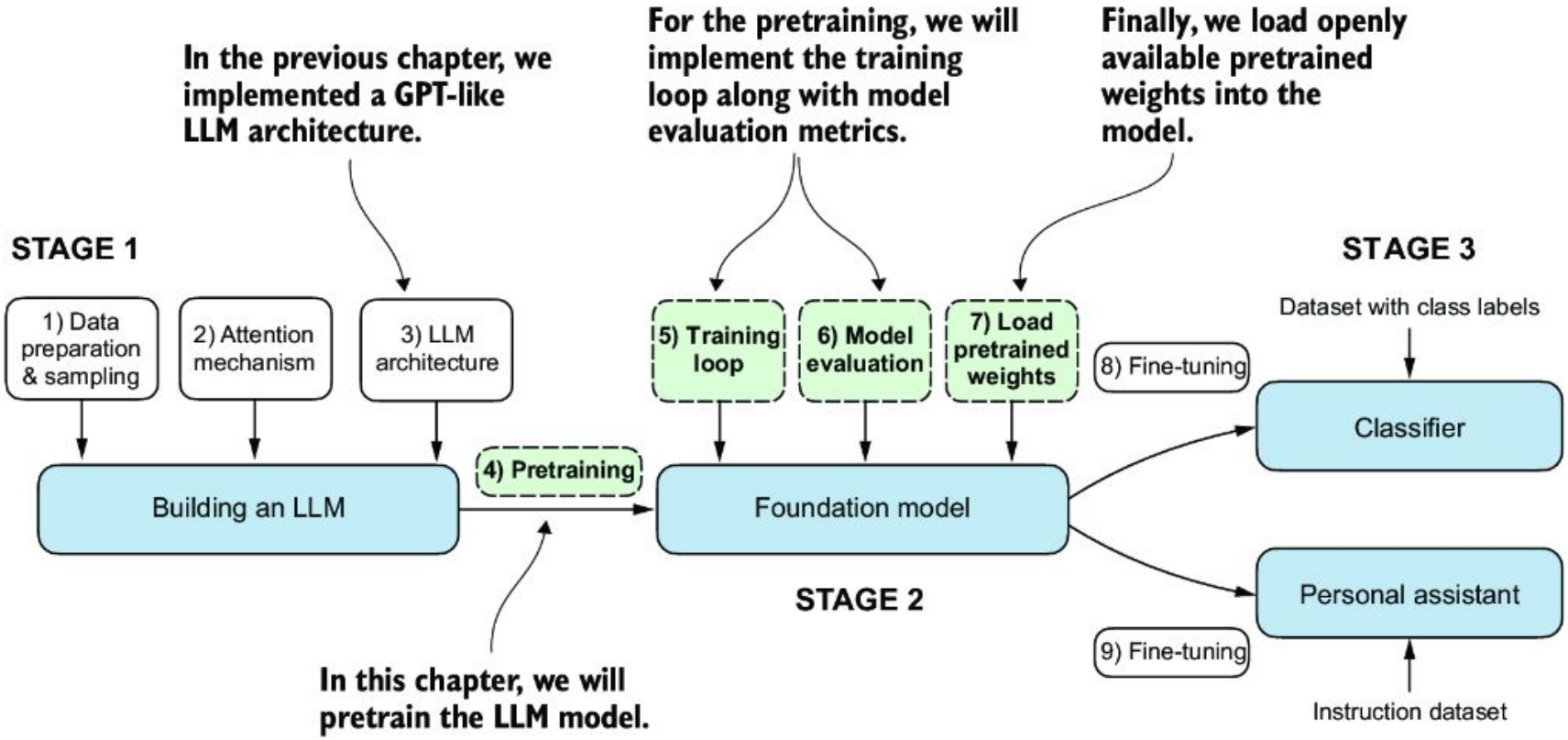
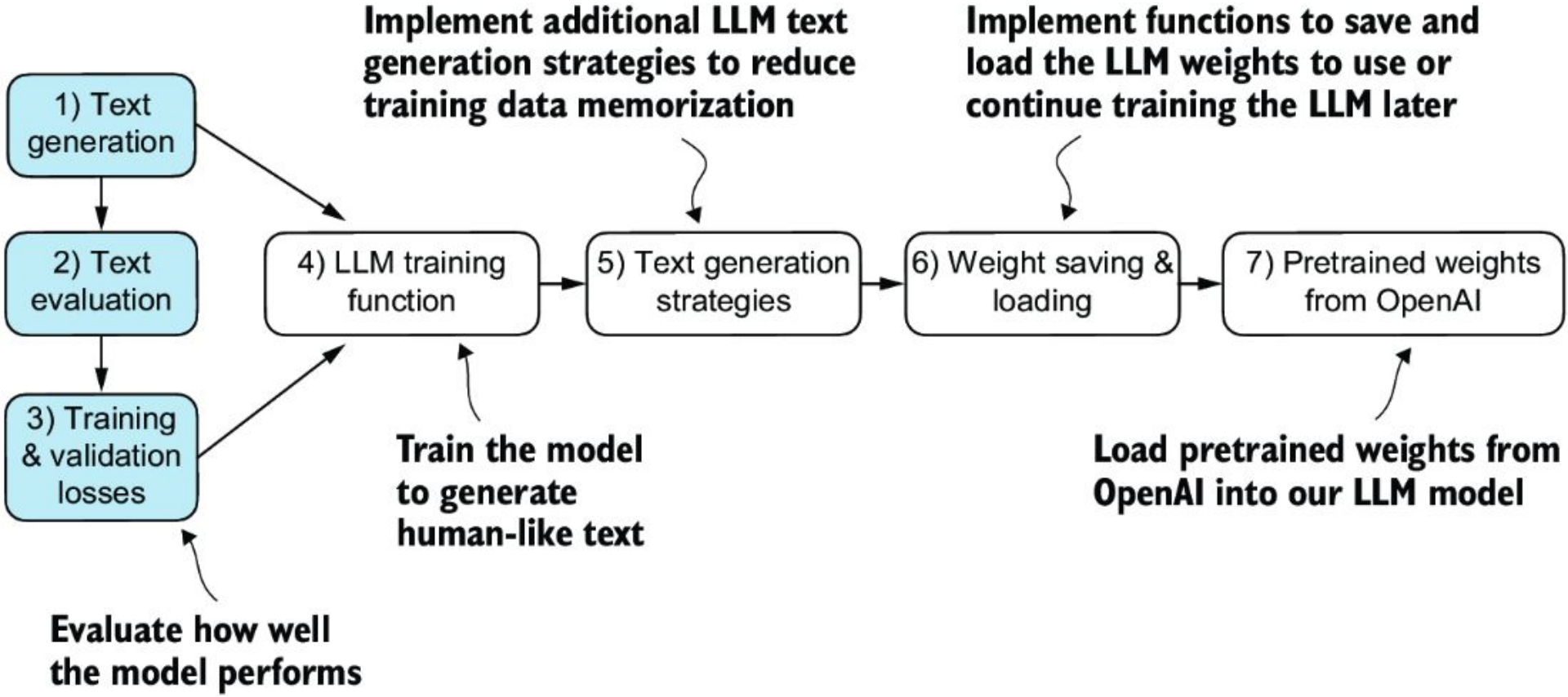
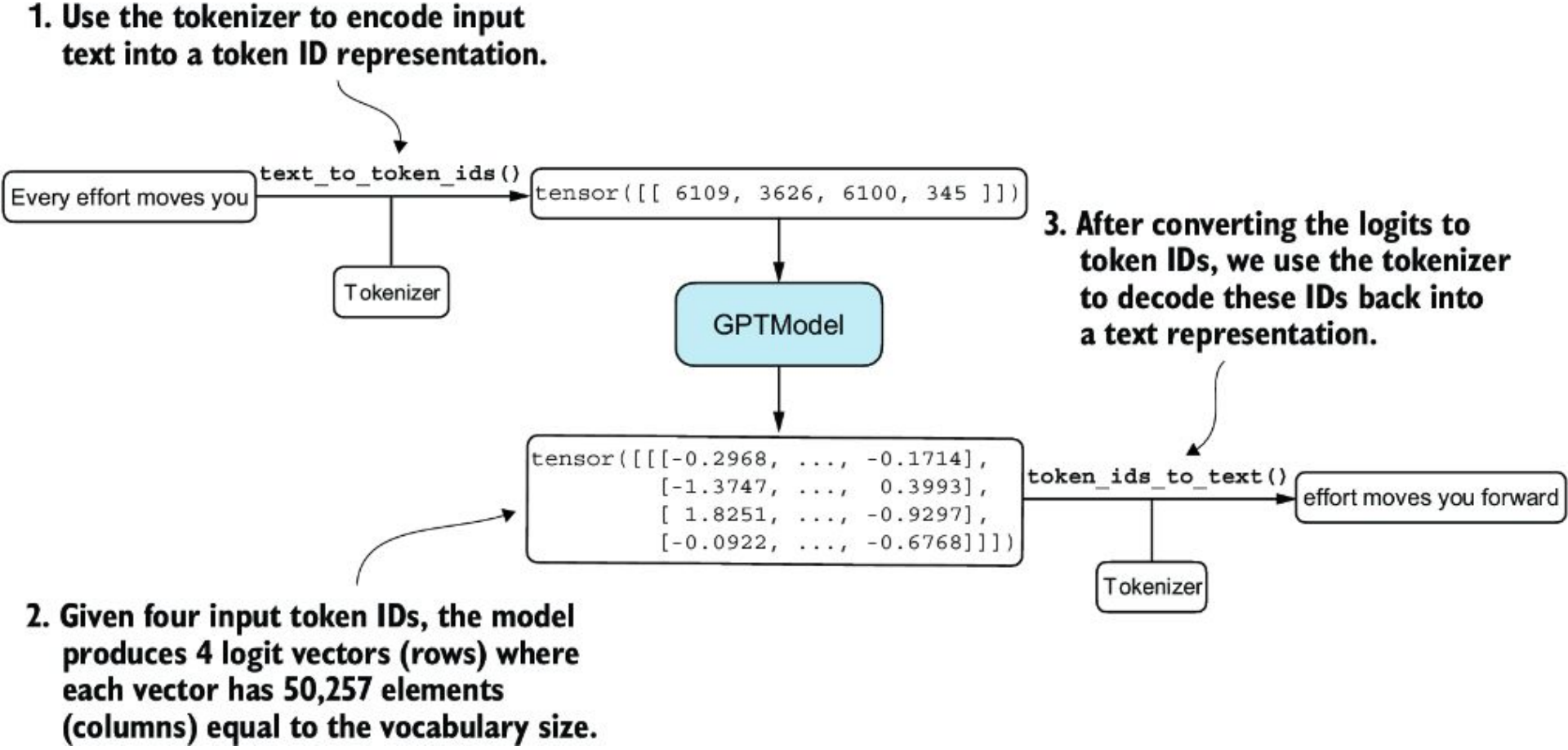
Pretraining on unlabeled data

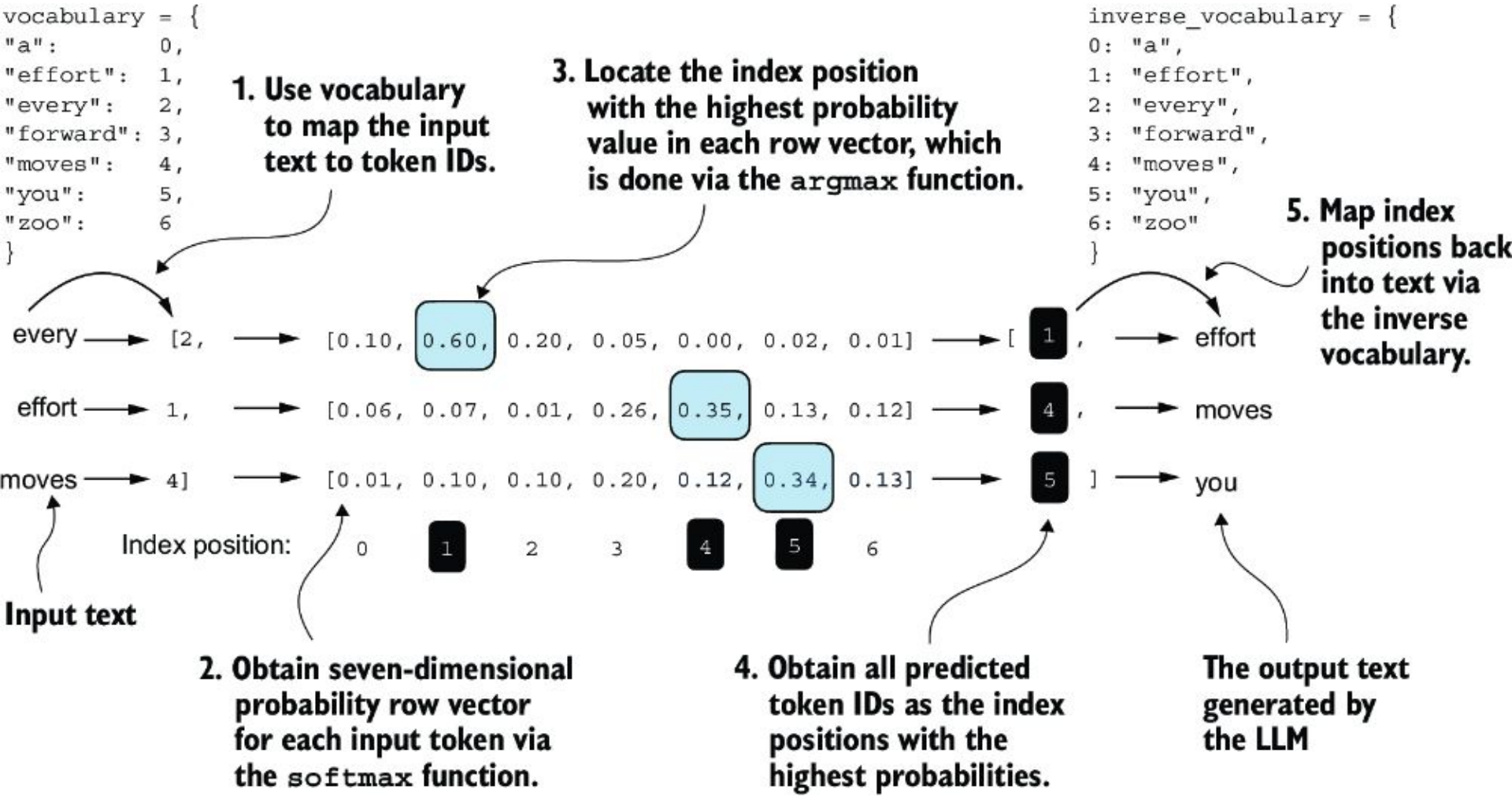
Dongyu Gong & Ping Luo
“Building LLMs from Scratch Workshop”
April 7, 2026

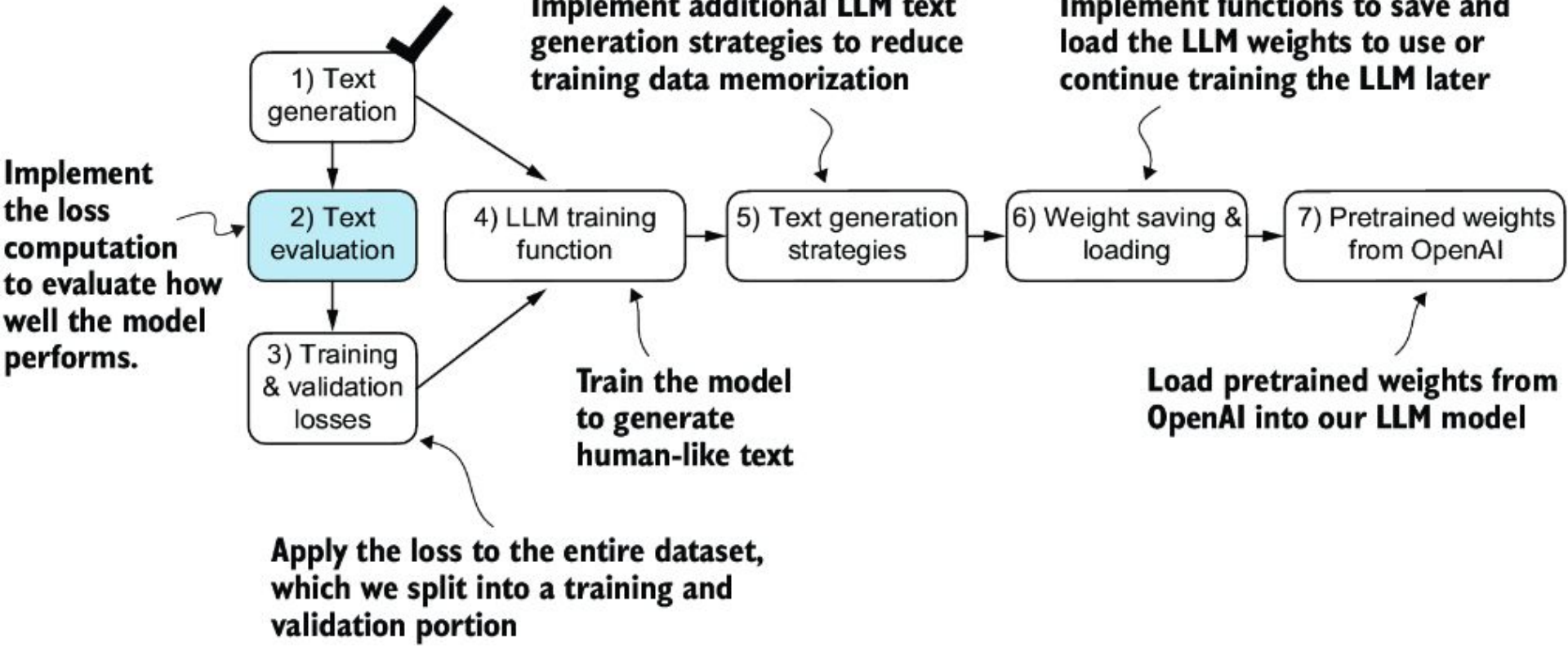
Roadmap

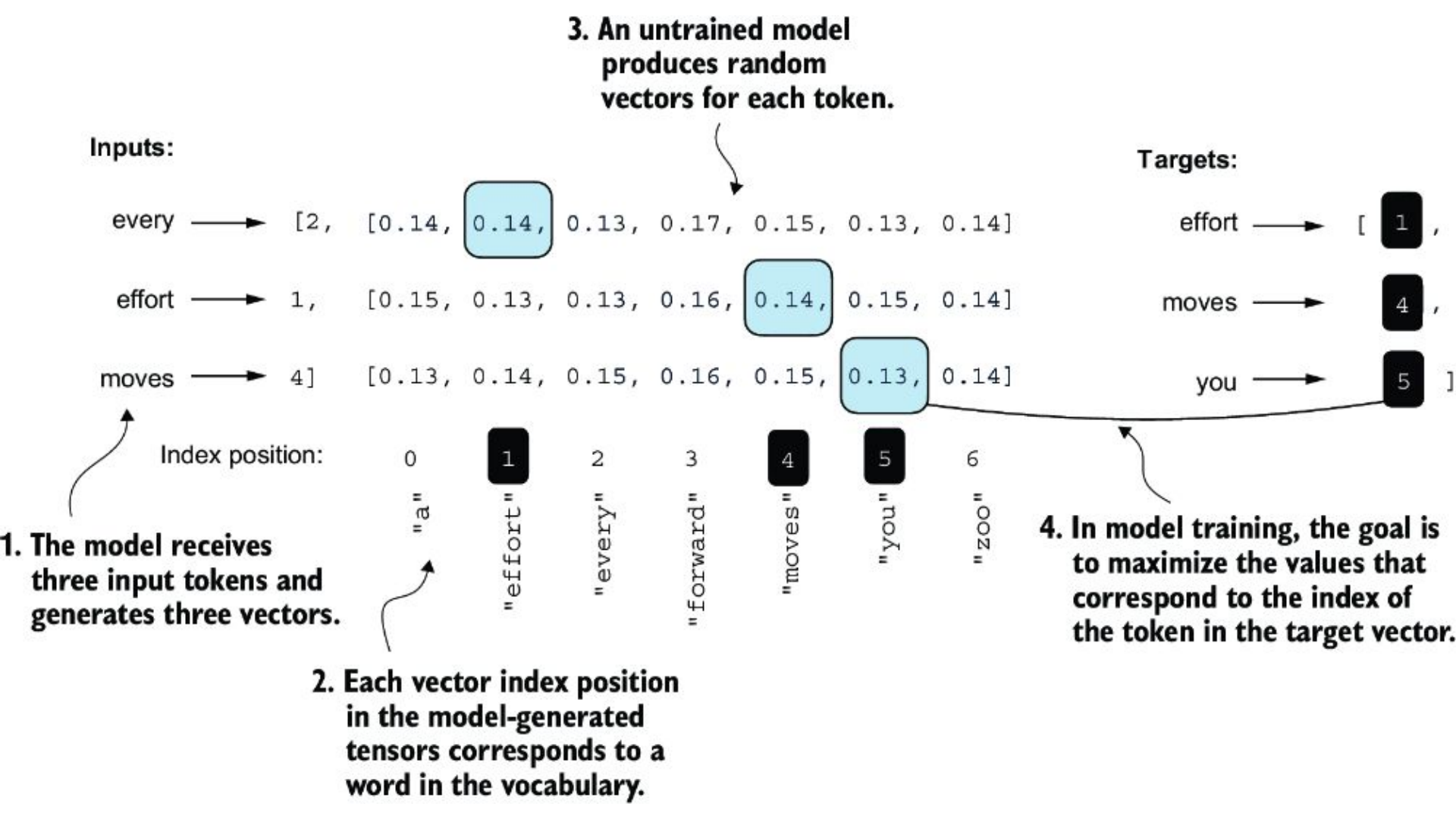

In the previous chapter, we implemented a GPT-like LLM architecture.
For the pretraining, we will implement the training loop along with model evaluation metrics.
Finally, we load openly available pretrained weights into the model.
STAGE 1
1) Data preparation & sampling
2) Attention mechanism
3) LLM architecture
Building an LLM
4) Pretraining
In this chapter, we will pretrain the LLM model.
5) Training loop
6) Model evaluation
7) Load pretrained weights
Foundation model
STAGE 2
8) Fine-tuning
STAGE 3
Dataset with class labels
Classifier
Personal assistant
9) Fine-tuning
Instruction dataset


Implement additional LLM text generation strategies to reduce training data memorization
Implement functions to save and load the LLM weights to use or continue training the LLM later
1) Text generation
2) Text evaluation
3) Training & validation losses
4) LLM training function
5) Text generation strategies
6) Weight saving & loading
7) Pretrained weights from OpenAI
Train the model to generate human-like text
Load pretrained weights from OpenAI into our LLM model
Evaluate how well the model performs


1. Use the tokenizer to encode input text into a token ID representation.
Every effort moves you
text_to_token_ids()
Tokenizer
tensor([[6109, 3626, 6100, 345]])
GPTModel
tensor([[[-0.2968, ..., -0.1714],
[-1.3747, ..., 0.3993],
[1.8251, ..., -0.9297],
[-0.0922, ..., -0.6768]]])
2. Given four input token IDs, the model produces 4 logit vectors (rows) where each vector has 50,257 elements (columns) equal to the vocabulary size.
3. After converting the logits to token IDs, we use the tokenizer to decode these IDs back into a text representation.
token_ids_to_text()
Tokenizer
effort moves you forward


vocabulary = {
"a": 0,
"effort": 1,
"every": 2,
"forward": 3,
"moves": 4,
"you": 5,
"zoo": 6
}
1. Use vocabulary to map the input text to token IDs.
3. Locate the index position with the highest probability value in each row vector, which is done via the argmax function.
inverse_vocabulary = {
0: "a",
1: "effort",
2: "every",
3: "forward",
4: "moves",
5: "you",
6: "zoo"
}
5. Map index positions back into text via the inverse vocabulary.
every → [2, → [0.10, 0.60, 0.20, 0.05, 0.00, 0.02, 0.01] → [1 , → effort
effort → 1, → [0.06, 0.07, 0.01, 0.26, 0.35, 0.13, 0.12] → 4 , → moves
moves → 4] → [0.01, 0.10, 0.10, 0.20, 0.12, 0.34, 0.13] → 5] → you
Index position: 0 1 2 3 4 5 6
Input text
2. Obtain seven-dimensional probability row vector for each input token via the softmax function.
4. Obtain all predicted token IDs as the index positions with the highest probabilities.
The output text generated by the LLM


1) Text generation
2) Text evaluation
3) Training & validation losses
4) LLM training function
5) Text generation strategies
6) Weight saving & loading
7) Pretrained weights from OpenAI
Implement additional LLM text generation strategies to reduce training data memorization
Implement functions to save and load the LLM weights to use or continue training the LLM later
Implement the loss computation to evaluate how well the model performs.
Train the model to generate human-like text
Load pretrained weights from OpenAI into our LLM model
Apply the loss to the entire dataset, which we split into a training and validation portion

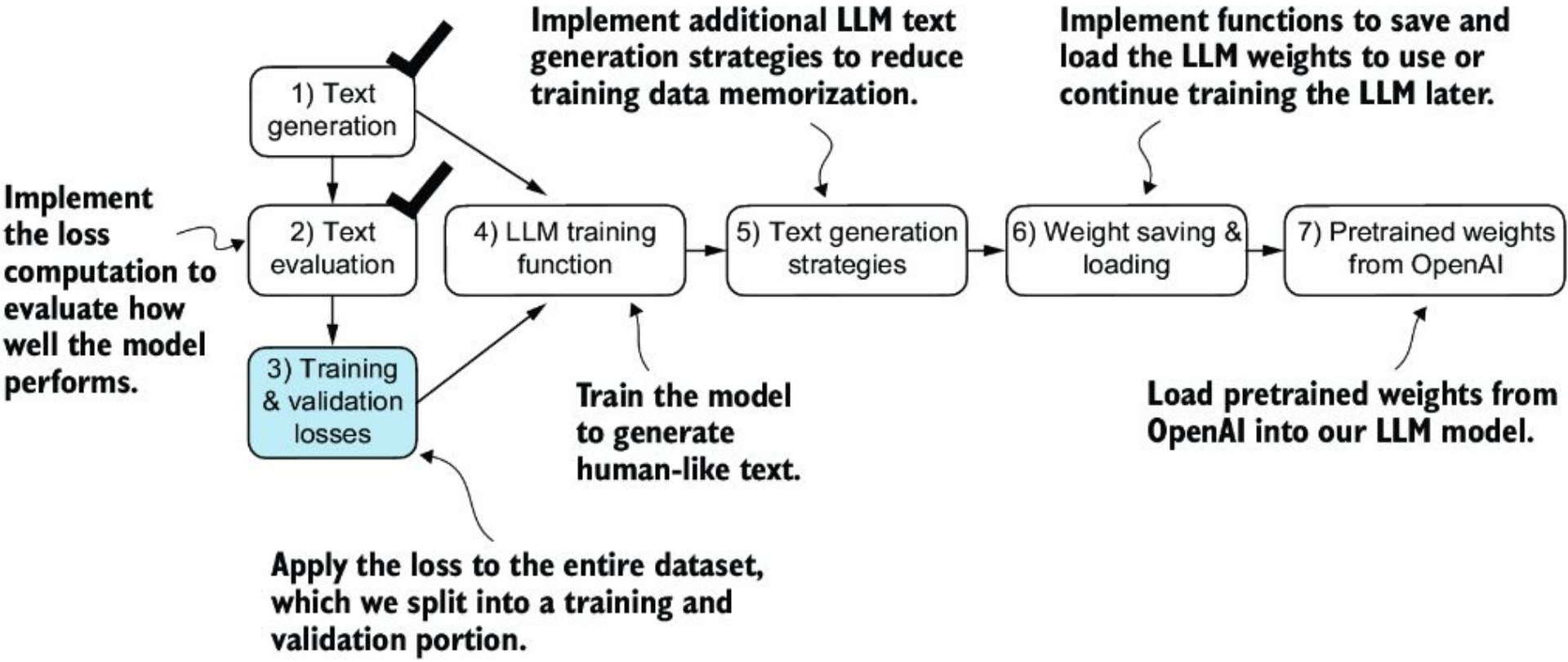

3. An untrained model produces random vectors for each token.
Inputs:
every → [2, [0.14, 0.14, 0.13, 0.17, 0.15, 0.13, 0.14]
effort → 1, [0.15, 0.13, 0.13, 0.16, 0.14, 0.15, 0.14]
moves → 4] [0.13, 0.14, 0.15, 0.16, 0.15, 0.13, 0.14]
Targets:
effort → [1 ,
moves → 4 ,
you → 5]
Index position: 0 1 2 3 4 5 6
"a" "effort" "every" "forward" "moves" "you" "zoo"
1. The model receives three input tokens and generates three vectors.
2. Each vector index position in the model-generated tensors corresponds to a word in the vocabulary.
4. In model training, the goal is to maximize the values that correspond to the index of the token in the target vector.

Cross-entropy Loss and Perplexity

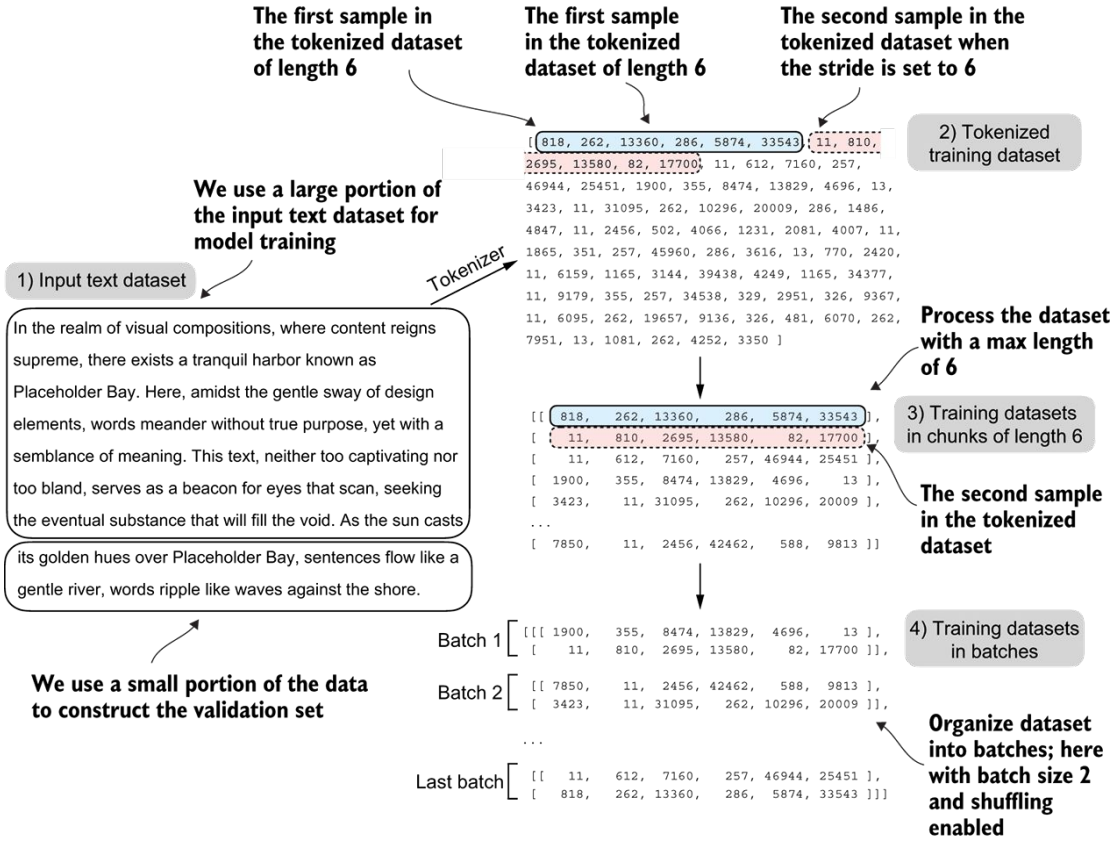
$$L_{CE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{C} y_{i,j}\log(\hat{y}_{i,j})$$

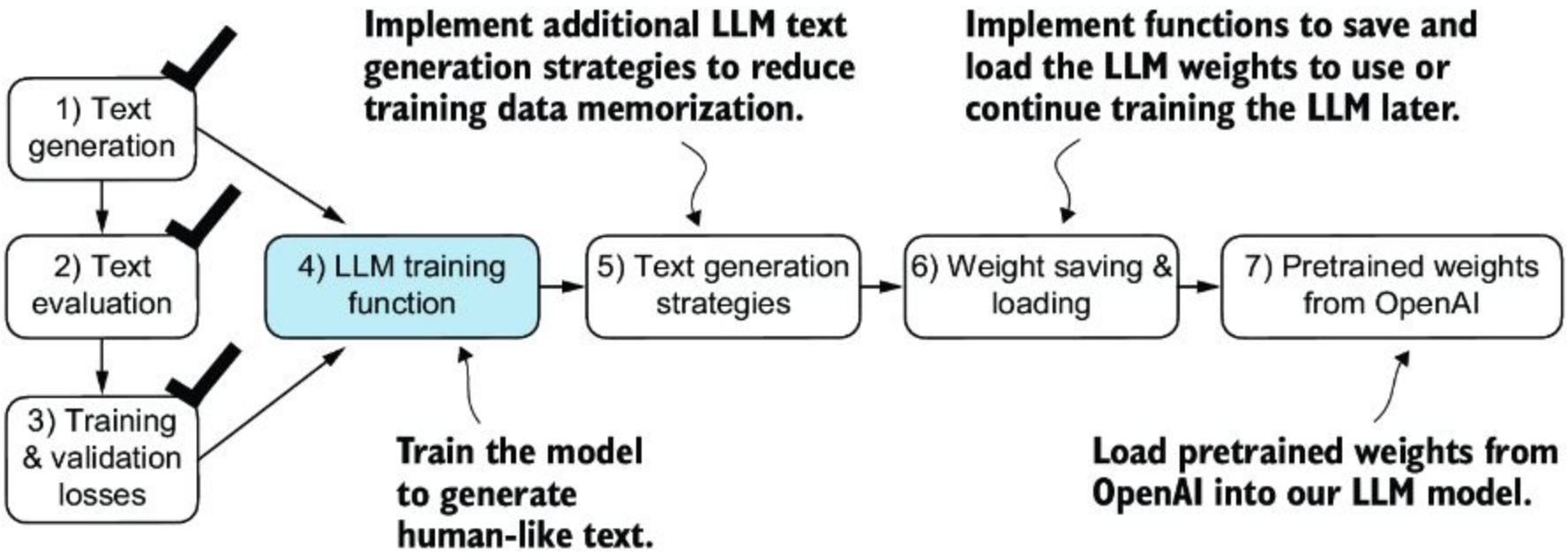
$$H(W) = -\frac{1}{T}\sum_{t=1}^{T}\log q(w_t|w_{<t})$$

$$PPL(W) = e^{H(W)}$$

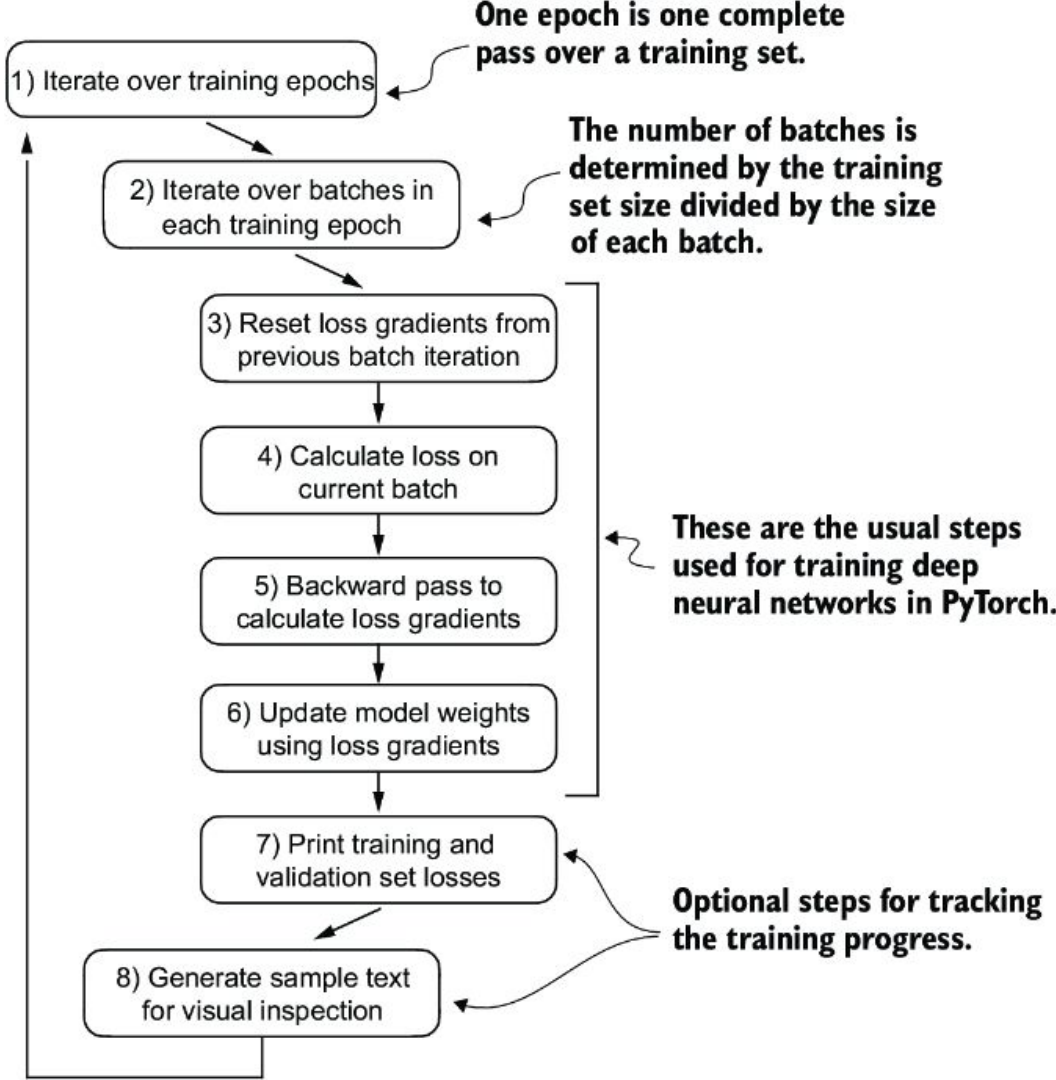

1) Text generation
2) Text evaluation
3) Training & validation losses
4) LLM training function
5) Text generation strategies
6) Weight saving & loading
7) Pretrained weights from OpenAI
Implement additional LLM text generation strategies to reduce training data memorization.
Implement functions to save and load the LLM weights to use or continue training the LLM later.
Implement the loss computation to evaluate how well the model performs.
Train the model to generate human-like text.
Load pretrained weights from OpenAI into our LLM model.
Apply the loss to the entire dataset, which we split into a training and validation portion.

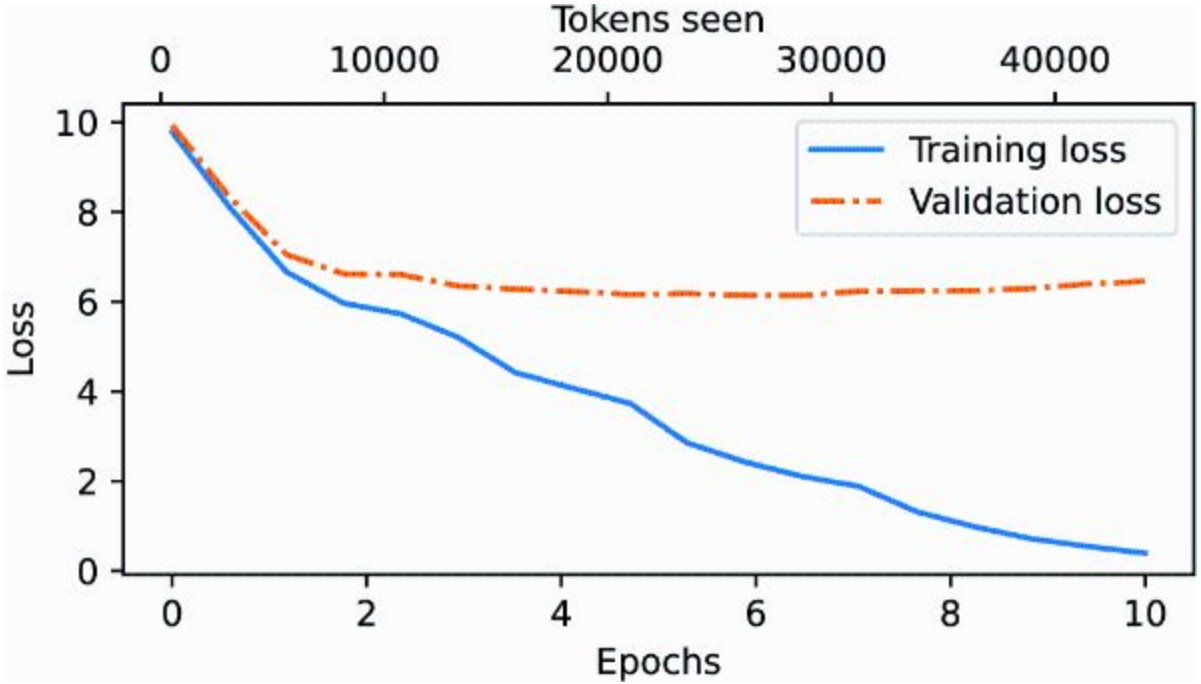
Training and validation loss

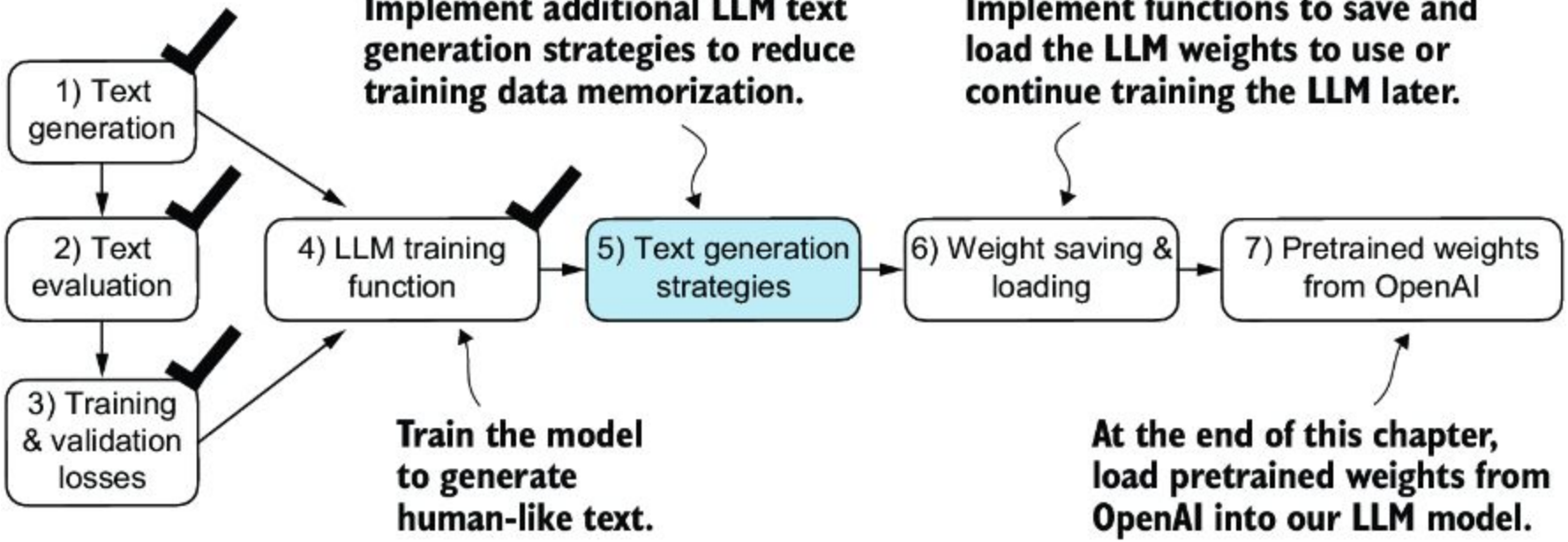



1) Text generation
2) Text evaluation
3) Training & validation losses
4) LLM training function
5) Text generation strategies
6) Weight saving & loading
7) Pretrained weights from OpenAI
Implement additional LLM text generation strategies to reduce training data memorization.
Implement functions to save and load the LLM weights to use or continue training the LLM later.
Train the model to generate human-like text.
Load pretrained weights from OpenAI into our LLM model.

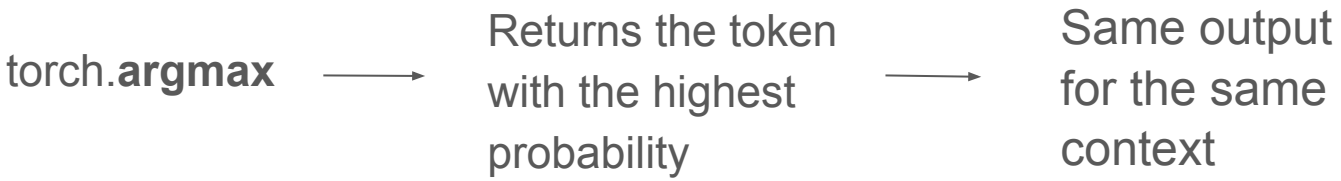
Training an LLM




Tokens seen
0
10000
20000
30000
40000
Training loss
Validation loss
Loss
10
8
6
4
2
0
0
2
4
6
8
10
Epochs


Implement additional LLM text generation strategies to reduce training data memorization.
Implement functions to save and load the LLM weights to use or continue training the LLM later.
1) Text generation
2) Text evaluation
3) Training & validation losses
4) LLM training function
5) Text generation strategies
6) Weight saving & loading
7) Pretrained weights from OpenAI
Train the model to generate human-like text.
At the end of this chapter, load pretrained weights from OpenAI into our LLM model.

Decoding strategies



Decoding strategies: text generation strategies that add randomness to generate more original text

Add probabilistic distribution

Adds a probabilistic selection process to the next token generation task

```
probas = torch.softmax(next_token_logits, dim=0)
next_token_id = torch.multinomial(probas, num_samples=1).item()
```

```
vocab = {
    "closer": 0,
    "every": 1,
    "effort": 2,
    "forward": 3,
    "inches": 4,
    "moves": 5,
    "pizza": 6,
    "toward": 7,
    "you": 8,
}
```

context = “every effort moves you"

73 x closer
0 x every
0 x effort
582 x forward
2 x inches
0 x moves
0 x pizza
343 x toward

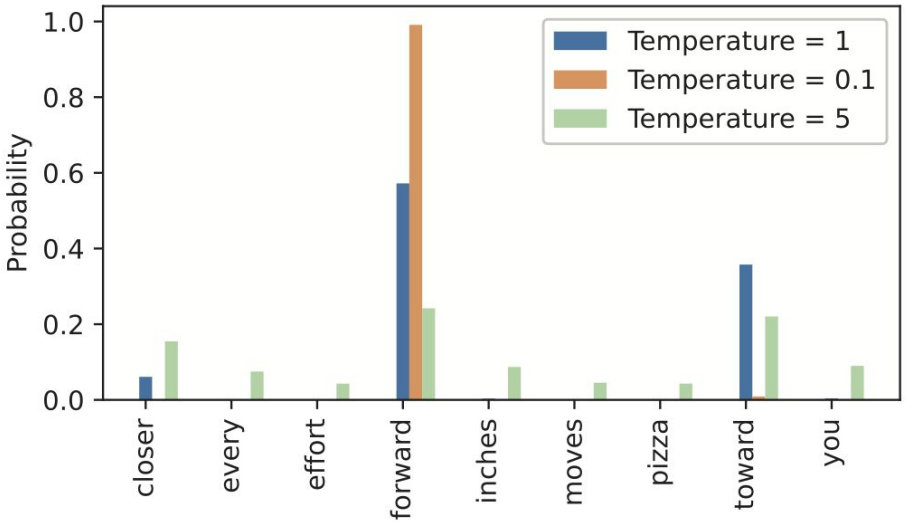
Sampling distribution after running multinominal selection for 1000 times

Temperature scaling

Divide the logits by a number >0 (**temperature**) to further control the distribution and selection process

“every effort moves you”

```
def softmax_with_temperature(logits, temperature):
    scaled_logits = logits / temperature
    return torch.softmax(scaled_logits, dim=0)
```



Top-k sampling

Mutlinomial function coupled with temperature scaling:

- Pros: provide increased diversity of the outputs
- Cons: may select a next token that breaks grammar or meaning.

Top-k sampling: restrict the sampled token to the top k most likely tokens and exclude all others from the selection process.

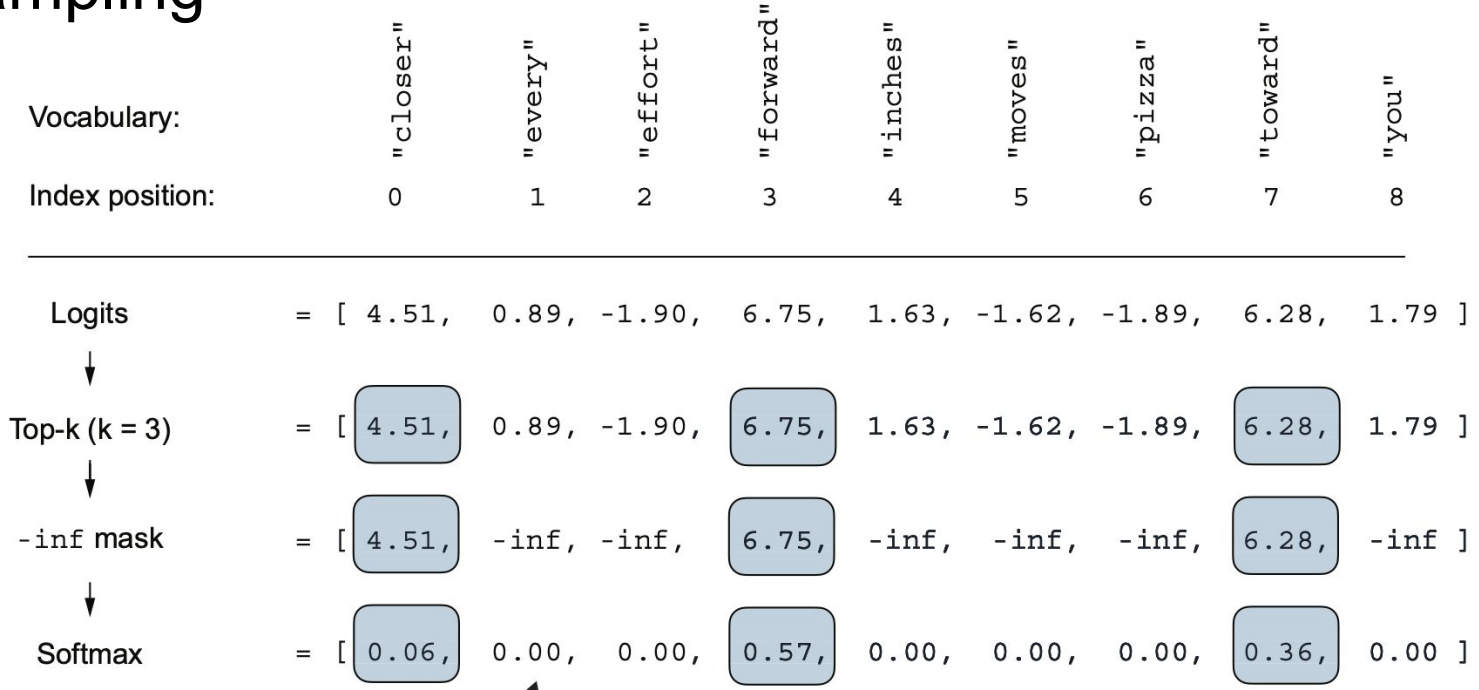
```
top_k = 3
top_logits, top_pos = torch.topk(next_token_logits, top_k)
new_logits = torch.where(
    condition=next_token_logits < top_logits[-1],
    input=torch.tensor(float('-inf')),
    other=next_token_logits
)
print(new_logits)
```

Identifies logits less than the minimum in the top 3

Assigns –inf to these lower logits

Retains the original logits for all other tokens

Top-k sampling



New text generation function

```
def generate(model, idx, max_new_tokens, context_size,
             temperature=0.0, top_k=None, eos_id=None):
    for _ in range(max_new_tokens):
        idx_cond = idx[:, -context_size:]
        with torch.no_grad():
            logits = model(idx_cond)
        logits = logits[:, -1, :]

        if top_k is not None:
            top_logits, _ = torch.topk(logits, top_k)
            min_val = top_logits[:, -1]
            logits = torch.where(
                logits < min_val,
                torch.tensor(float('-inf')).to(logits.device),
                logits
            )
        if temperature > 0.0:
            logits = logits / temperature
            probs = torch.softmax(logits, dim=-1)
            idx_next = torch.multinomial(probs, num_samples=1)
        else:
            idx_next = torch.argmax(logits, dim=-1, keepdim=True)
        if idx_next == eos_id:
            break
        idx = torch.cat((idx, idx_next), dim=1)
    return idx
```

The for loop is the same as before: gets logits and only focuses on the last time step.

Filters logits with top_k sampling

Applies temperature scaling

Carries out greedy next-token selection as before when temperature scaling is disabled

Stops generating early if end-of-sequence token is encountered

Loading and saving model weights

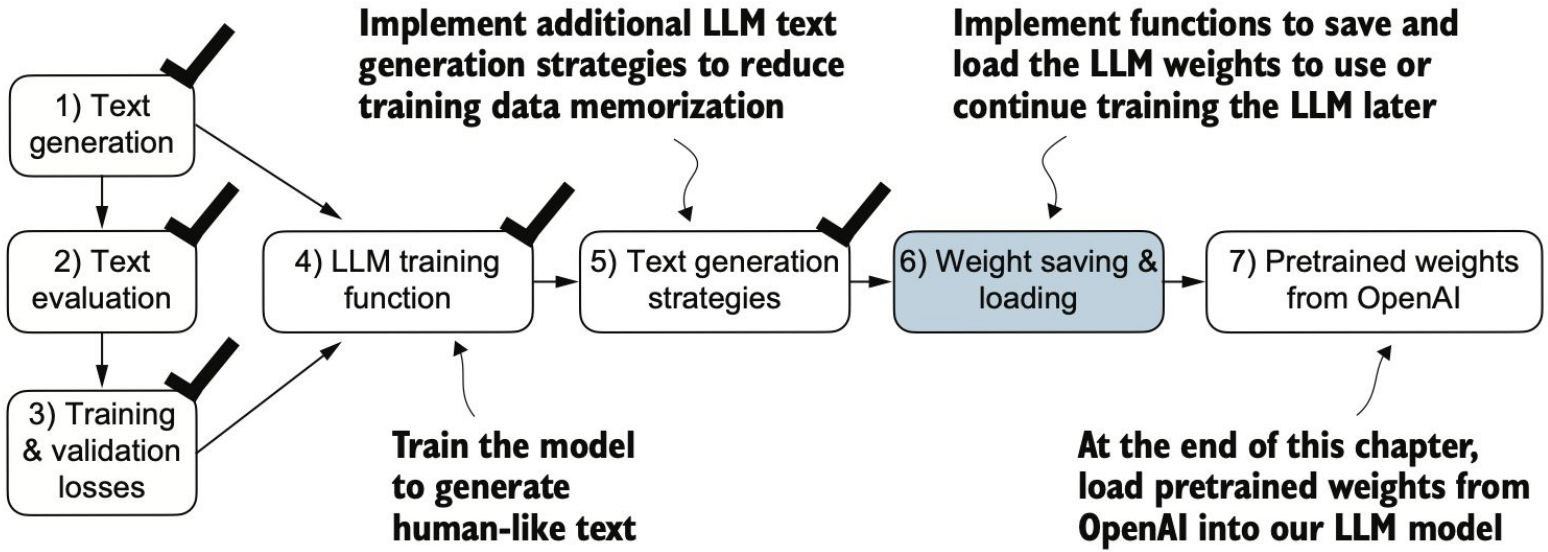


Figure 5.16 After training and inspecting the model, it is often helpful to save the model so that we can use or continue training it later (step 6).

Save a model and load it for reference

Saving

```
torch.save(model.state_dict(), "model.pth")
```

state_dict: a dictionary mapping each layer to its parameters

Loading

```
model = GPTModel(GPT_CONFIG_124M)
model.load_state_dict(torch.load("model.pth", map_location=device))
model.eval()
```

Save a model and load it to continue pre-training

Saving

```
torch.save({
    "model_state_dict": model.state_dict(),
    "optimizer_state_dict": optimizer.state_dict(),
    },
    "model_and_optimizer.pth")
```

Loading

```
checkpoint = torch.load("model_and_optimizer.pth", map_location=device)

model = GPTModel(GPT_CONFIG_124M)
model.load_state_dict(checkpoint["model_state_dict"])
optimizer = torch.optim.AdamW(model.parameters(), lr=5e-4, weight_decay=0.1)
optimizer.load_state_dict(checkpoint["optimizer_state_dict"])

model.train();
```

Load pretrained weights from OpenAI

Weights are saved via TensorFlow, which is required to load the weights.

Use a boilerplate code to download the files from OpenAI: gpt_download.py

```
from gpt_download import download_and_load_gpt2
settings, params = download_and_load_gpt2(
  model_size="124M", models_dir="gpt2"
)
```

Transfer weights from params to GPTModel instance, including

Positional and token embedding weights, multi-layter transformer blocks (q, k, v for attention weights and biases, feed forward, norm1, norm2, out_project)

Different sizes of GPT-2 models